City-G: Publisher-Blind, Time-Blind End-to-End Encryption for Massive Asynchronous Groups

City-G Working Group

Version 0.1.1
February 11, 2026

Abstract

Large-group end-to-end encrypted messaging is difficult to scale without giving up either security guarantees or operational simplicity. Existing approaches often force strict global ordering for key schedule evolution, rely on wall-clock assumptions, or expose revocation-side metadata that degrades privacy at scale. This paper presents City-G, a protocol profile designed for high-concurrency, asynchronous environments with millions of members and strict server-side blindness over message secrets.

City-Gcombines a time-blind forward-secrecy schedule, deterministic acceptance, and a post-revocation secrecy (PRS) barrier based on a KEM-tree cover. The design separates public acceptance state from client secret state, enforces canonical wire encoding and raw-byte commitment discipline, and introduces explicit fail-closed rules for malformed or adversarial updates. We describe the protocol architecture, state machine, and security properties; analyze concurrency and scalability behavior; and compare City-Gwith mainstream group key management patterns. The result is a protocol that keeps steady-state operations lightweight while making revocation handling explicit, verifiable, and robust under active-server threat assumptions.

1 Introduction

End-to-end encrypted (E2EE) group systems must simultaneously satisfy three pressures: security under active attackers, asynchronous operability for offline devices, and production-grade scalability under bursty concurrency. In large deployments, those pressures collide. Systems that optimize for simplicity often centralize trust in servers for key distribution. Systems that optimize for strict cryptographic structure frequently impose serialization points that become throughput bottlenecks. Systems that optimize for throughput may under-specify edge-case validation, enabling interop drift or latent security regressions.

City-Gtargets this gap: robust security semantics without a server-side message secret role, explicit acceptance determinism without wall-clock dependence, and practical behavior for high-concurrency populations. The profile analyzed here is the final unified specification `v0.1.1` (FS-hybrid + PRS barrier), in which forward secrecy, acceptance rules, and post-revocation secrecy are specified in a single normative document [4].

Core thesis. City-Gis useful because it treats security invariants as state-machine invariants, not just crypto primitive selections. In practice, compromise and drift happen at boundaries: encoding,

ordering, replay, and recovery behavior. CITY-Gcodifies those boundaries with explicit MUST-level rules and verifiable transitions.

2 Problem Setting and Design Goals

2.1 Operational setting

We consider very large asynchronous groups where:

- clients may be offline for long intervals,
- network delivery is adversarial and reordering is normal,
- joins and regular sends occur concurrently at high rates,
- revocations are infrequent but high impact,
- acceptance infrastructure must scale horizontally.

2.2 Threat model

The adversary may control transport and publishers, reorder or replay traffic, and attempt structural malleation. Excluded or revoked members may collude with compromised devices. Store-now-decrypt-later adversaries are in scope. The acceptance server is not trusted with message secrets and is treated as potentially curious; selected checks also model an active malicious server.

2.3 Goals

1. **Confidentiality without server message secrets:** server cannot decrypt payloads or header-protected message material.
2. **Time-blind forward secrecy:** acceptance must not rely on wall clocks.
3. **Post-revocation secrecy:** once revocation roots change, barrier advancement is required before normal traffic resumes.
4. **Deterministic acceptance:** canonical encoding, closed-world headers, bounded maxima, and fail-closed checks.
5. **Concurrency at scale:** preserve high steady-state throughput for non-revocation anchors.

3 System Model and State Separation

3.1 Roles

- **Clients:** hold long-lived and evolving secrets, produce anchors, encrypt/decrypt payloads.
- **Acceptance server:** validates anchors and manages public state transitions.
- **Membership/SRX subsystem:** provides revocation and join enumerations with integrity.

3.2 Anchor types

City-Gdefines three anchor types: `JOIN`, `REGULAR`, and `MERGE`. Barrier update material (key 175) and merge-only keys force `MERGE` typing; join leaf key publication (key 177) forces `JOIN`. The profile uses a closed-world key registry with strict mode-specific presence rules [4].

3.3 State separation

A critical design choice is separating:

- **Public state** (server): acceptance counters, per-device chain cursors, barrier public tree commitment, revocation hash commitment.
- **Secret state** (client): FS chain secret, epoch derivations, barrier key, private KEM keys on self path.

This separation prevents accidental "server helpfulness" from becoming a secrecy dependency.

4 Cryptographic and Encoding Discipline

4.1 Primitive suite

The profile uses BLAKE3-based domain-separated hashing, HKDF-BLAKE3 with fixed output length 32, ChaCha20-Poly1305 AEAD, and ML-KEM-768 for cover operations. Deterministic CBOR (RFC 8949 profile) is mandatory for all critical structures [2, 6–8].

4.2 Why canonical encoding is first-class

In distributed cryptographic systems, parser-level ambiguity is often as dangerous as primitive breakage. City-Gtherefore requires deterministic re-encoding checks and maps failures to explicit codes. It also distinguishes semantic objects from their raw transmitted bytes where identity is digest-bound (e.g., barrier update bytes).

5 Protocol Overview

5.1 FS-hybrid schedule

Each anchor carries a monotone epoch counter `fs_ec`. Clients evolve local FS chain secret `K_fs` and derive per-epoch material (including `tau_e`) via profile-fixed labels. The profile binds device-level chain commitments to barrier state by defining:

$$\begin{aligned}\texttt{fs\_dev\_commit} = H_L(&\texttt{"fs/dev/chain/v2"},\\ &[\texttt{header[108]}, \texttt{header[141]}, \texttt{header[152]},\\ &\texttt{header[176]}, \texttt{barrier\_update\_digest}]).\end{aligned} \tag{1}$$

This construction ties device progression to barrier progression.

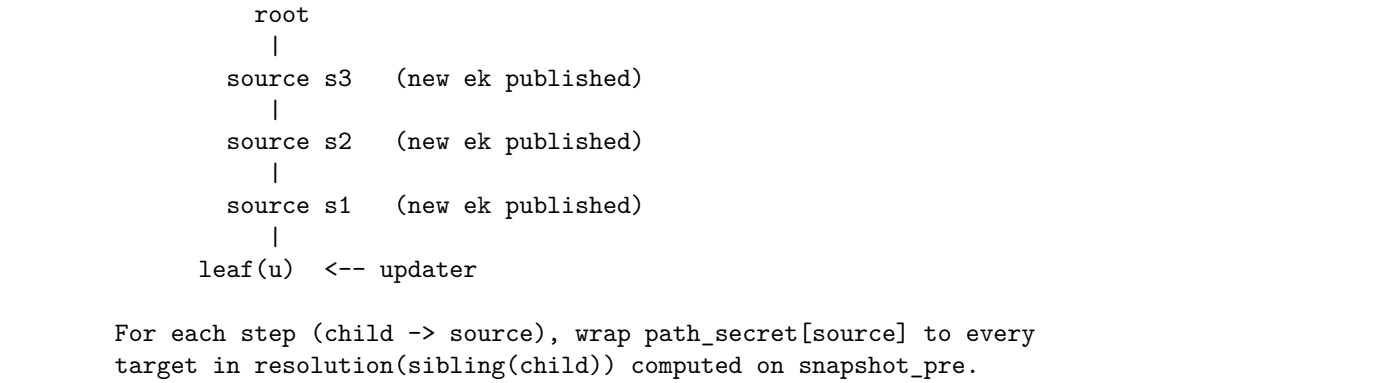

Figure 1: KEM-tree cover intuition: on-path secrets are wrapped to copath resolution targets (non-blank nodes) to enable unique-match recovery.

5.2 Payload envelope and nonce safety

Payload ciphertexts are carried in a 3-field envelope with explicit `msg_index`. Message key and nonce derivation include `msg_index`; uniqueness is mandatory for each fixed identifier tuple {gid, weid, t, xk_hash, E_k, barrier_version}. State enforcing uniqueness must be crash-safe.

5.3 Time-blind acceptance and FLG

Acceptance never uses wall-clock checks. Instead, forward-leap guards (FLG) bound `fs_ec`jumps against persisted acceptance state and policy-derived windows. This is a key scaling and reliability property: skewed clocks cannot silently widen exposure windows.

5.4 PRS barrier overview

The barrier is a committed public KEM tree with client-held private keys along self paths. Revocation updates are represented by barrier updates that include:

- pre- and post-update tree commitments,
- deterministic new internal public keys on updater path ancestors,
- node ciphertexts that transport ancestor path secrets to copath resolutions.

Intuition. A barrier update moves secret material up the updater's direct path and "fans out" each on-path secret to the non-blank leaves in the sibling subtree (the copath resolution). Concretely, for each on-path source node, the updater encapsulates to every non-blank public key in the corresponding resolution set and wraps the 32-byte `path_secret[source]` under the resulting KEM shared secret.

Matching hint. Each node ciphertext carries a 16-byte `target_pk_hash` prefix to cheaply filter candidates. Correctness and security do not depend on this prefix being collision-resistant: the full 32-byte public-key hash is bound into AEAD AAD during unwrap, so false matches fail closed. Clients recover a unique matching ciphertext, derive upward secrets, compute new `K_barrier`, and update local key material.

5.5 Updater and recover semantics

The updater is explicitly excluded from running the generic recover path for its own update. Instead, it must persist pending activation state before publish, correlate accepted bytes by digest, then activate. This avoids false-local failures and crash-induced divergence.

5.6 SRX trigger semantics

The final profile makes SRX activation explicit and state-based: on MERGE anchors, SRX is required exactly when the current revocation-roots digest differs from the pivot digest (derived from persisted barrier state), and forbidden otherwise. This closes a class of ambiguous implementations where SRX could be over-applied or skipped silently. Operationally, SRX logic and barrier revocation gating are complementary: SRX controls proof obligations under revocation-root transitions, while barrier gating controls key-state advancement.

Acceptance skeleton for a barrier-carrying merge. For anchors carrying `barrier_update`, the final profile requires structural/canonical/hash-chain barrier checks before expensive proof verification, then fail-fast proof verification, then atomic commit.

1. Parse deterministic header map; reject unknown keys or invalid maxima.
2. Enforce anchor-type presence matrix and closed-world policy.
3. Verify FS base, device-chain commit, and FLG bounds.
4. Parse and deterministic-check `BarrierUpdate` and `KemTreeCoverPayload`.
5. Validate path structure, canonical ordering, and exact `ExpectedNodeSet`.
6. Build `snapshot_pre`; verify `kem_tree_hash_before`.
7. Apply `CP.new_public_keys`; verify `kem_tree_hash_after`.
8. Validate `ExpectedPairs` completeness/minimality.
9. Verify proofs in fail-fast order; then atomically commit public state.

6 Concurrency and Scalability Behavior

6.1 Steady-state path

Under stable revocation roots, `JOIN` and `REGULAR` anchors do not require barrier update bytes and do not bump barrier version. This keeps high-frequency traffic on a lightweight path while preserving deterministic acceptance.

6.2 Revocation synchronization point

When revocation roots change, City-Gintroduces an explicit synchronization requirement: the next admissible transition must be a merge carrying barrier update and barrier version increment. This is intentional. It localizes revocation cost to a cold path and prevents accidental post-revocation plaintext recoverability under stale barrier keys.

Table 1: Asymptotic cost profile (high-level)

Operation	Dominant work	Notes
Join/regular acceptance	Header/proof checks + O(1) state writes	Fast path when revocation roots unchanged
Barrier merge acceptance	$O(N_{\max})$ tree/auth work + $O((J+R)\cdot h)$ path blanking + $O(P)$ pair validation	Cold path; cost scales with tree size and revocation coverage
Client recover (non-updater)	$O(P)$ candidate scan + one decap + $O(h)$ upward derivation	Fail-closed on 0 or multi-match
Updater rekey	$O(h)$ path-secret derivation + $O(P)$ KEM encaps/wrap operations	Burst cost concentrated on revocation events

6.3 Large-group complexity

Let $h = \log_2(N_{\max})$, $J = |\texttt{JoinSet}|$, $R = |\texttt{RevokedLeafSet}|$, and $P = |\texttt{ExpectedPairs}|$.

The design choice is clear: optimize daily throughput; pay structured complexity only when revocation semantics demand it.

Concrete scale example. If a deployment provisions $N_{\max} = 2^{20}$ (about 1,048,576 leaves), then $h = 20$. A full barrier merge necessarily touches the whole public tree for snapshot authentication and commitment recomputation ($O(N_{\max})$ hashing) and then performs per-join/per-revocation path blanking work on the order of $(J+R)\cdot h$. The ciphertext fanout term P is workload-dependent: in the worst case where almost all leaves are non-blank, P can approach $O(N_{\max})$; in typical operation it is smaller. The key point is that this burst cost is paid only on revocation-root transitions, not on daily anchors.

7 Security Analysis

7.1 Property matrix

7.2 Active-server resistance

Server-side validation is necessary but not sufficient against a malicious server controlling public-tree responses. CITY-Gaddresses this by requiring updater and full clients to authenticate fetched public tree snapshots against previously committed hashes before proceeding. Snapshot auth failures are explicit diagnostics (960.9).

7.3 Why the barrier gate matters

Without mandatory barrier advancement on revocation-root change, senders could continue deriving payload keys under stale barrier context. The profile now forbids that path by acceptance gating. This is the main reason post-revocation secrecy is a protocol property rather than an application convention.

Table 2: Security and protocol properties in the final profile

Property	Status	Mechanism
Confidentiality	Achieved	End-to-end payload keys from client-held `tau_e`and `K_barrier`; server keyless for message decrypt path
Authentication and integrity	Achieved	Anchor authentication + deterministic validation + proof commitments
Forward secrecy (time-blind)	Achieved	Evolving FS chain, zeroization guidance, no wall-clock acceptance dependency
Post-revocation secrecy	Achieved (policy-enforced)	Mandatory barrier update gating when revocation roots change
Membership consistency	Achieved	Commitment chain, deterministic tree/hash checks, ExpectedPairs rules
Asynchronicity and scalability	Achieved	Fast path for non-revocation anchors; explicit cold path for revocations
Concurrency	Achieved with bounded retries	Deterministic acceptance and barrier version gating; retries on races are explicit
Post-compromise recovery	Conditional	Requires compromise to cease and subsequent correct evolution/update
Deniability	Out of scope in this profile	Strong sender authentication is explicit design choice

7.4 Proactive PCS refresh and FS reseed

The final profile also allows a proactive barrier update mode (`barrier_update_reason = pcs_refresh`) when revocation roots are unchanged, subject to strict time-blind rate limits. This provides a practical post-compromise recovery path for a healed endpoint without waiting for a revocation event. At activation, clients reseed the FS chain with the newly recovered barrier key ($K_{fs} \leftarrow \text{HKDF}(K_{fs} \| K_{\text{barrier\_new}}, \ldots)$), so recovery benefits propagate to both barrier- and FS-derived message schedules.

This does not create "magic PCS" against an attacker who still controls an authorized member state; instead, it provides MLS-like recovery semantics once compromise of that member state has ceased and a valid refresh is accepted.

8 Comparison with Related Approaches

8.1 Compared with MLS-style trees

Messaging Layer Security (MLS) standardizes tree-based group key management with strong formal structure [1]. CITY-Gdiffers in emphasis:

- explicit time-blind acceptance and FLG-based epoch control,
- explicit server-keyless message path and raw-byte digest discipline,

- revocation-root to barrier-update gating as a mandatory acceptance condition, plus policy-governed proactive PCS refresh,
- separate treatment of fast-path anchors and revocation cold path.

Both systems share the same fundamental group PCS boundary: if an attacker still controls an authorized member state, that attacker can continue to follow group updates. Recovery occurs after a healthy self-update/refresh is accepted. This is not a "better/worse" claim in the abstract; it is a design fit claim for high-concurrency asynchronous environments that prioritize deterministic acceptance behavior, explicit revocation semantics, and policy-governed refresh cadence.

8.2 Compared with sender-key fanout designs

Sender-key systems can provide excellent throughput but often centralize revocation complexity in server policy or out-of-band repair. City-Gcodifies revocation transition rules in the protocol state machine itself, reducing reliance on implicit operational assumptions.

9 Implementation Guidance

9.1 Non-negotiable engineering invariants

- Never re-encode digest-bound raw objects before identity checks.
- Treat deterministic encoding as a security property, not formatting hygiene.
- Persist uniqueness and anti-replay state crash-safely before use.
- Keep updater pending-state activation digest-correlated and atomic.
- Keep pairwise $(dk_t, pkhash_t)$ updates crash-safe.

9.2 Concurrency policy for proactive refresh

Proactive PCS refresh is intentionally treated as a cold-path operation. To prevent traffic amplification and retry storms, the profile enforces time-blind rate limits in acceptance: per-device minimum epoch distance, group-wide minimum epoch distance, and a one-winner-per-slot rule in epoch space. Implementations should pair these checks with jittered client backoff after rate-limit rejections.

Operationally, this keeps high-frequency JOIN/REGULAR traffic on the lightweight path while making compromise-recovery refreshes explicit, auditable, and bounded.

9.3 Verification levels: full chain-check vs recover-only clients

The profile supports two verification levels for PRS barrier processing [4]. The difference is not about whether recovery can succeed, but about what classes of active-server behavior can be detected locally.

- **FULL-verifying clients** can fetch authenticated historical public-tree snapshots. The snapshot-fetch endpoint is:
  `FetchBarrierPublicTree`.

They perform mandatory chain-checks on `kem_tree_hash_before` and `kem_tree_hash_after`. FULL clients also verify deterministically derived internal public keys where derivable on their self path, providing strong protection against public-tree equivocation.

- **Recover-only clients** may lack snapshot fetch capability (e.g., due to storage or API restrictions). They still enforce deterministic encoding/canonicalization checks, path validation, fail-closed unique-match recovery, and deterministic local key-storage rules, but cannot locally authenticate historical public-tree snapshots.

Deployments SHOULD clearly label which client tiers can claim FULL verification, and snapshot-fetch endpoints SHOULD be authenticated and audited.

9.4 Storage optimization: `pkhash_t` instead of full `ek_t`

The profile requires clients to store $(dk_t, pkhash_t)$ pairs on self-path nodes, where `pkhash_t = H_pk(ek_t)`. This design is materially smaller than persisting full ML-KEM public keys per stored node: 32 bytes for `pkhash_t` instead of 1184 bytes for `ek_t`. The optimization keeps match/AAD validation deterministic while reducing persistent footprint and write amplification, which matters for constrained devices and high-update groups.

9.5 Why crash-safety is a cryptographic requirement

Three crash-safety rules are security-critical, not optional operational hygiene. First, `msg_index` uniqueness state must be durable before encryption to prevent nonce/key schedule reuse after restart. Second, client updates to $(dk_t, pkhash_t)$ must be atomic so match/AAD construction cannot drift into incoherent states. Third, updater persist-before-publish and digest-correlated activation are required to avoid local key-state rollback or split-brain activation under crash/restart races. Without these guarantees, confidentiality and post-revocation properties can be violated by implementation behavior even when primitives are sound.

9.6 Migration from FS-only profile

A practical migration plan should proceed in phases:

1. add header registry and state schema for barrier fields,
2. implement acceptance checks and error mapping in shadow mode,
3. ship barrier client code paths behind feature gates,
4. validate KATs and mixed offline/online recovery scenarios,
5. enforce revocation-change gating in production.

9.7 Test strategy

The profile's mandatory KAT set should be treated as conformance gates, not documentation examples. In addition to vector tests, teams should run:

- parser differential tests for deterministic encoding rejection,

- fault-injection on updater crash/restart windows,
- adversarial tests for malformed ExpectedPairs and multi-match recovery.
- CI builds should fail on spec/whitepaper/bibliography version mismatches and on undefined citations after bibtex/pdflatex passes.

10 Limitations and Future Work

- $N_{\max}$ is fixed for the lifetime of a group in this profile; deployments must provision tree capacity up front.
- Barrier merges can become heavy under large, bursty revocation sets; operational budgeting remains important.
- Full verification depends on authenticated access to historical public tree states.
- Formal verification artifacts can be expanded beyond present conformance/KAT structure.
- Deniability is not a target of this profile due to explicit sender authentication requirements.

Future work includes deeper formalization of liveness under sustained adversarial churn, adaptive cover packing strategies, and reference interoperability suites across independent implementations.

11 Conclusion

City-Gdemonstrates that large-scale asynchronous E2EE can be both security-rigorous and operations-conscious when acceptance semantics are treated as first-class cryptographic boundaries. The protocol's contribution is not a single primitive, but a coherent composition: time-blind FS progression, deterministic acceptance, explicit revocation-to-barrier gating, and client-verifiable cold-path recovery.

For deployments where revocation correctness, server blindness, and high concurrency are all non-negotiable, this profile offers a practical and auditable path forward.

A Normative Profile Reference

This whitepaper describes the final unified profile documented in:

`docs/specs.md` (City-GUnified Spec, v0.1.1 final).

The specification remains normative; this paper is explanatory. Normative requirement-language conventions in the specification follow RFC terminology conventions [3, 5].

B Notation Summary

Symbol / term	Meaning
`fs_ec`	Epoch counter carried on anchors
`tau_e`	Per-epoch FS-derived secret material
`K_barrier`	Barrier key bound to barrier version and revocation roots
`ExpectedNodeSet`	Required internal-node publication set on updater path ancestors
`ExpectedPairs`	Required ciphertext pair coverage from on-path source to copath resolution

References

[1] Richard Barnes, Benjamin Beurdouche, Raphael Robert, Jon Millican, Christopher Patton, and Casper Wood. The Messaging Layer Security (MLS) Protocol. RFC 9420, RFC Editor, July 2023. URL https://www.rfc-editor.org/rfc/rfc9420.

[2] Carsten Bormann and Paul Hoffman. Concise Binary Object Representation (CBOR). RFC 8949, RFC Editor, December 2020. URL https://www.rfc-editor.org/rfc/rfc8949.

[3] Scott Bradner. Key words for use in RFCs to Indicate Requirement Levels. RFC 2119, RFC Editor, March 1997. URL https://www.rfc-editor.org/rfc/rfc2119.

[4] City-G Working Group. CITY-G Unified Spec (FS-HYBRID + PRS BARRIER), v0.1.1, February 2026. Normative protocol specification in repository: docs/specs.md.

[5] Benjamin Leiba. Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. RFC 8174, RFC Editor, May 2017. URL https://www.rfc-editor.org/rfc/rfc8174.

[6] National Institute of Standards and Technology. Module-Lattice-Based Key-Encapsulation Mechanism Standard. FIPS 203, U.S. Department of Commerce, August 2024. URL https://doi.org/10.6028/NIST.FIPS.203.

[7] Yaron Nir and Adam Langley. ChaCha20 and Poly1305 for IETF Protocols. RFC 8439, RFC Editor, June 2018. URL https://www.rfc-editor.org/rfc/rfc8439.

[8] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. BLAKE3: one function, fast everywhere, 2020. URL https://github.com/BLAKE3-team/BLAKE3-specs/blob/master/blake3.pdf.